

運用 Spanner 和 Vertex AI 執行相似度搜尋

來源：<https://codelabs.developers.google.com/codelabs/similarity-search-spanner-vertex?hl=zh-tw>

步驟數：8

依據使用者輸入內容推薦服飾的相似度搜尋應用程式，並針對儲存在 Spanner 中的資料執行搜尋，並由 Vector Search 建立索引，以最鄰近的項目回應。

目錄

1. [簡介](#)
2. [需求條件](#)
3. [後端：建立 Spanner 資料來源和嵌入內容](#)
4. [工作流程工作：將 Spanner 資料匯出至 Vector Search](#)
5. [部署 Vector Search 索引](#)
6. [前端：使用者資料到向量搜尋](#)
7. [清除所用資源](#)
8. [結論](#)

1. 簡介

深度學習的最新進展，讓我們得以用能擷取語意的方式表示文字和其他資料。因此我們開發了新的搜尋方式，稱為向量搜尋，使用文字的向量表示法 (稱為嵌入) 找出與使用者查詢最相關的文件。對於服飾搜尋等應用程式，向量搜尋比傳統搜尋更合適，因為使用者通常會依說明、風格或脈絡搜尋項目，而非依確切的產品或品牌名稱搜尋。我們可以將 Cloud Spanner 資料庫與 Vector Search 整合，執行向量相似度比對。客戶可以同時使用 Spanner 和 [Vector Search](#)，建立強大的整合功能，結合 Spanner 的可用性、可靠性和規模，以及 Vertex AI Vector Search 的進階相似度搜尋功能。這項搜尋作業會比較 Vector Search 搜尋索引中的項目 [嵌入](#)，並傳回最相似的結果。

用途

假設您是時裝零售商的資料科學家，正努力跟上快速變遷的趨勢、產品搜尋和推薦內容。但您資源有限，且資料孤島林立。這篇網誌文章說明如何使用服飾資料的相似性搜尋方法，實作服飾推薦用途。涵蓋的步驟如下：

1. 從 Spanner 取得的資料
2. 使用 ML.PREDICT 為服飾資料生成的向量，並儲存在 Spanner 中
3. 使用 Dataflow 和工作流程工作，將 Spanner 向量資料整合至 Vector Search
4. 執行向量搜尋，找出使用者輸入內容的相似比對結果

我們將建構示範網頁應用程式，根據使用者輸入內容執行服飾搜尋。使用者可以輸入文字說明，透過應用程式搜尋服飾。

從 Spanner 匯出至 Vector Search 索引：

服飾搜尋的資料會儲存在 Spanner 中。我們將直接從 Spanner 資料，在 ML.PREDICT 建構中叫用 Vertex AI [Embeddings](#) API。接著，我們會運用 Dataflow 和 Workflow 工作，將這項資料 (目錄和嵌入) 大量上傳至 Vertex AI 的 Vector Search，並重新整理索引。

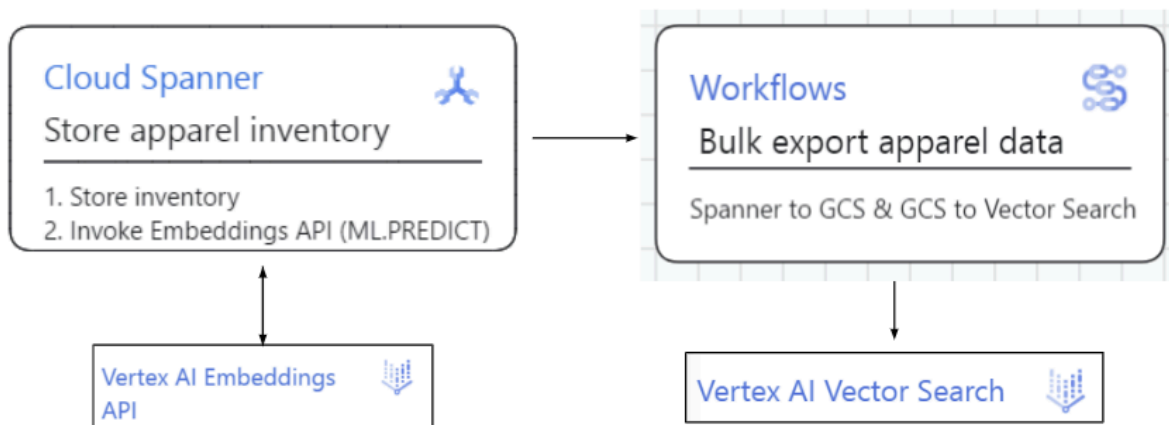
在索引上執行使用者查詢：

使用者輸入服飾說明後，應用程式會使用 Text Embeddings API 即時生成嵌入內容。然後將這項資訊做為 Vector Search API 的輸入內容，從索引中找出 10 個相關的產品說明，並顯示對應的圖片。

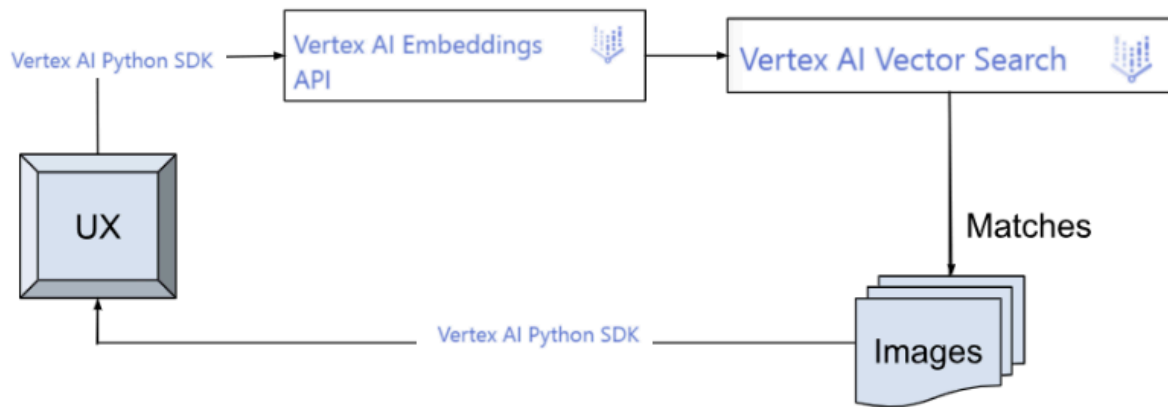
架構總覽

下圖分為兩部分，顯示 Spanner 向量搜尋應用程式的架構：

從 Spanner 建立 Vector Search 索引：



用戶端應用程式，用於在索引上執行使用者查詢：



建構項目

從 Spanner 匯出至向量索引：

- Spanner 資料庫，用於儲存及管理來源資料和對應的嵌入內容
- 工作流程工作，可將資料 (ID 和嵌入) 大量上傳至 Vertex AI Vector Search 資料庫。
- 用來從索引中尋找相關產品說明的 Vector Search API。

在索引上執行使用者查詢：

- 這個網頁應用程式可讓使用者輸入服飾的文字說明，並使用已部署的索引端點執行相似度搜尋，然後傳回與輸入內容最接近的服飾。

運作方式

使用者輸入服飾的文字說明後，網頁應用程式會將說明傳送至 Vector Search API。接著，Vector Search API 會使用服飾說明的嵌入內容，從索引中找出最相關的產品說明。接著，系統會向使用者顯示產品說明和相應圖片。一般工作流程如下所示：

1. 為 Spanner 中儲存的資料生成嵌入。
 2. 將嵌入項目匯出並上傳至 Vector Search 索引。
 3. 執行最鄰近搜尋，查詢 Vector Search 搜尋索引中的類似項目。
-

2. 需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
- 已啟用計費功能的 Google Cloud 雲端專案

事前準備

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)
3. 確認已[啟用](#)所有必要 API (Cloud Spanner、Vertex AI、Google Cloud Storage)
4. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境，已預先載入 gcloud。如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

5. 使用進行中的 Google Cloud 雲端專案前往 [Cloud Spanner](#) 頁面，即可開始使用
-

步驟 3 · 約 0 分鐘

3. 後端：建立 Spanner 資料來源和嵌入內容

在這個用途中，Spanner 資料庫會存放服飾的庫存，以及對應的圖片和說明。請務必為文字說明生成嵌入，並以 `ARRAY<float64>` 格式儲存在 Spanner 資料庫中。

1. 建立 Spanner 資料

建立名為「spanner-vertex」的執行個體，以及名為「spanner-vertex-embeddings」的資料庫。使用 DDL 建立資料表：

```
CREATE TABLE
  apparels ( id NUMERIC,
            category STRING(100),
            sub_category STRING(50),
            uri STRING(200),
            content STRING(2000),
            embedding ARRAY<FLOAT64>
          )
PRIMARY KEY
  (id);
```

2. 使用 INSERT SQL 將資料插入資料表

如要插入範例資料的指令碼，請參閱[這篇文章](#)。

3. 建立文字嵌入模型

這是必要步驟，因為我們需要為輸入內容生成嵌入。以下是相同項目的 DDL：

```
CREATE MODEL text_embeddings INPUT(content STRING(MAX))
OUTPUT(
  embeddings
  STRUCT<
    statistics STRUCT<truncated BOOL, token_count FLOAT64>,
    values ARRAY<FLOAT64>>
  )
REMOTE OPTIONS (
  endpoint = '//aiplatform.googleapis.com/projects/abis-345004/locations/us-
centrall/publishers/google/models/textembedding-gecko');
```

4. 為來源資料生成文字嵌入

建立資料表來儲存嵌入，並插入生成的嵌入。在實際的資料庫應用程式中，載入至 Spanner 的資料 (最多到步驟 2) 會是交易式資料。為確保設計最佳做法不受影響，我偏好讓交易資料表正規化，因此會為嵌入建立個別資料表。

```
CREATE TABLE apparels_embeddings (id string(100), embedding ARRAY<FLOAT64>)
PRIMARY KEY (id);

INSERT INTO apparels_embeddings(id, embeddings)
SELECT CAST(id as string), embeddings.values
FROM ML.PREDICT(
  MODEL text_embeddings,
  (SELECT id, content from apparels)
) ;
```

大量內容和嵌入項目已準備就緒，現在讓我們建立 Vector Search 索引和端點，儲存有助於執行 Vector Search 的嵌入項目。

4. 工作流程工作：將 Spanner 資料匯出至 Vector Search

1. 建立 Cloud Storage bucket

這是將 Spanner 的嵌入內容以 JSON 格式儲存在 GCS bucket 的必要步驟，因為向量搜尋會將這些內容視為輸入。在與 Spanner 資料相同的區域中[建立 bucket](#)。視需要建立資料夾，但主要是在其中建立名為 `empty.json` 的空白檔案。

2. 設定 Cloud Workflow

如要設定從 Spanner 批次匯出至 Vertex AI Vector Search 索引，請按照下列步驟操作：

建立空白索引：

請確認向量搜尋索引與 Cloud Storage 值區和資料位於相同區域。在管理索引頁面的「為批次更新建立索引」部分，按照控制台分頁標籤下的 11 個步驟操作。在傳遞至 `contentsDeltaUri` 的資料夾中，建立名為 `empty.json` 的空白檔案，因為您必須有這個檔案才能建立索引。這會建立空白索引。

如果已有索引，可以略過這個步驟。工作流程會覆寫索引。

注意：您無法將空白索引部署至端點。因此，我們將部署至端點的步驟延後，先將向量資料匯出至 Cloud Storage。

複製這個 Git 存放區：複製 Git 存放區的方法有很多種，其中一種是使用 [GitHub CLI](#) 執行下列指令。在 Cloud Shell 終端機執行下列 2 個指令：

```
gh repo clone cloudspannerecosystem/spanner-ai

cd spanner-ai/vertex-vector-search/workflows
```

這個資料夾包含兩個檔案

- `batch-export.yaml`：這是工作流程定義。
- `sample-batch-input.json`：這是工作流程輸入參數的範例。

從範例檔案設定 `input.json`：首先，複製範例 JSON。

```
cp sample-batch-input.json input.json
```

然後編輯 `input.json`，填寫專案詳細資料。在本例中，您的 JSON 應如下所示：

```
{
  "project_id": "<<YOUR_PROJECT>>",
  "location": "<<us-central1>>",
  "dataflow": {
    "temp_location": "gs://<<YOUR_BUCKET>>/<<FOLDER_IF_ANY>>/workflow_temp"
  },
  "gcs": {
    "output_folder": "gs://<<YOUR_BUCKET>>/<<FOLDER_IF_ANY>>/workflow_output"
  },
  "spanner": {
    "instance_id": "spanner-vertex",
    "database_id": "spanner-vertex-embeddings",
    "table_name": "apparels_embeddings",
    "columns_to_export": "embedding,id"
  },
  "vertex": {
    "vector_search_index_id": "<<YOUR_INDEX_ID>>"
  }
}
```

設定權限

在正式版環境中，我們強烈建議您建立新的服務帳戶，並授予一或多個 IAM 角色，其中包含管理服務所需的最低權限。如要設定工作流程，將 Spanner (嵌入) 的資料匯出至 Vector Search 索引，您需要下列角色：

[Cloud Workflow 服務帳戶](#)：

根據預設，這項服務會使用 **Compute Engine 預設服務帳戶**。

如果您使用手動設定的服務帳戶，必須包含下列角色：

如要觸發 Dataflow 工作，您必須具備 **Dataflow 管理員**或 **Dataflow 工作者**角色。

如要模擬 Dataflow 工作站服務帳戶，請使用「服務帳戶使用者」。

如要寫入記錄，請使用「記錄寫入者」。

如要觸發 Vertex AI Vector Search 重建作業，請**成為 Vertex AI 使用者**。

[Dataflow 工作站服務帳戶](#)：

如果您使用手動設定的服務帳戶，必須包含下列角色：

如要管理資料流，請使用 **Dataflow 管理員**或 **Dataflow 工作者**角色。如要從 Spanner 讀取資料，請使用 **Cloud Spanner 資料庫讀取者**。所選 GCS Container Registry 的寫入權限：**GCS Storage Bucket Owner**。

3. 部署 Cloud Workflow

將工作流程 YAML 檔案部署至 Google Cloud 雲端專案。執行工作流程時，您可以設定要執行的區域或位置。

```
gcloud workflows deploy vector-export-workflow --source=batch-export.yaml --location="us-central1" [--service account=<service_account>]
```

or

```
gcloud workflows deploy vector-export-workflow --source=batch-export.yaml --location="us-central1"
```

現在，您應該可以在 Google Cloud 控制台的「Workflows」頁面中看到工作流程。

注意：您也可以透過 Google Cloud 控制台建立及部署工作流程。按照 Cloud 控制台中的提示操作。複製並貼上 batch-export.yaml 的內容，做為工作流程定義。

完成後，請執行工作流程，開始匯出資料。

4. 執行 Cloud Workflow

執行下列指令來執行工作流程：

```
gcloud workflows execute vector-export-workflow --data="$(cat input.json)"
```

執行作業應會顯示在「Workflows」的「執行作業」分頁中。這會將資料載入 Vector Search 資料庫並建立索引。

注意：您也可以使用「執行」按鈕從控制台執行。按照提示操作，並複製及貼上自訂 input.json 的內容做為輸入內容。

步驟 5 · 約 0 分鐘

5. 部署 Vector Search 索引

將索引部署至端點

如要部署索引，請按照下列步驟操作：

1. 在「Vector Search indexes」(向量搜尋索引) 頁面中，您應該會看到在上一個步驟 2 中建立的索引旁邊有「DEPLOY」(部署) 按鈕。或者，您也可以前往索引資訊頁面，然後按一下「DEPLOY TO ENDPOINT」(部署至端點) 按鈕。
2. 提供必要資訊，並將索引部署至端點。

或者，您也可以查看這個筆記本，[將模型部署至端點](#) (略過筆記本的部署部分)。部署完成後，請記下部署的索引 ID 和端點網址。

6. 前端：使用者資料到向量搜尋

讓我們建構簡單的 Python 應用程式，並使用 Gradio 支援的使用者體驗，快速測試實作項目：您可以參閱[這裡](#)的實作項目，在自己的 Colab 筆記本中實作這個試用版應用程式。

1. 我們會使用 aiplatform Python SDK 呼叫 Embeddings API，並叫用 Vector Search 索引端點。

```
# [START aiplatform_sdk_embedding]
!pip install google-cloud-aiplatform==1.35.0 --upgrade --quiet --user

import vertexai
vertexai.init(project=PROJECT_ID, location="us-central1")

from vertexai.language_models import TextEmbeddingModel

import sys
if "google.colab" in sys.modules:
    # Define project information
    PROJECT_ID = " " # Your project id
    LOCATION = " " # Your location

    # Authenticate user to Google Cloud
    from google.colab import auth
    auth.authenticate_user()
```

2. 我們將使用 Gradio 示範如何透過使用者介面，快速輕鬆地建構 AI 應用程式。請先重新啟動執行階段，再實作這個步驟。

```
!pip install gradio
import gradio as gr
```

3. 在使用者輸入內容後，從網頁應用程式叫用 Embeddings API，我們會使用文字嵌入模型：textembedding-gecko@latest

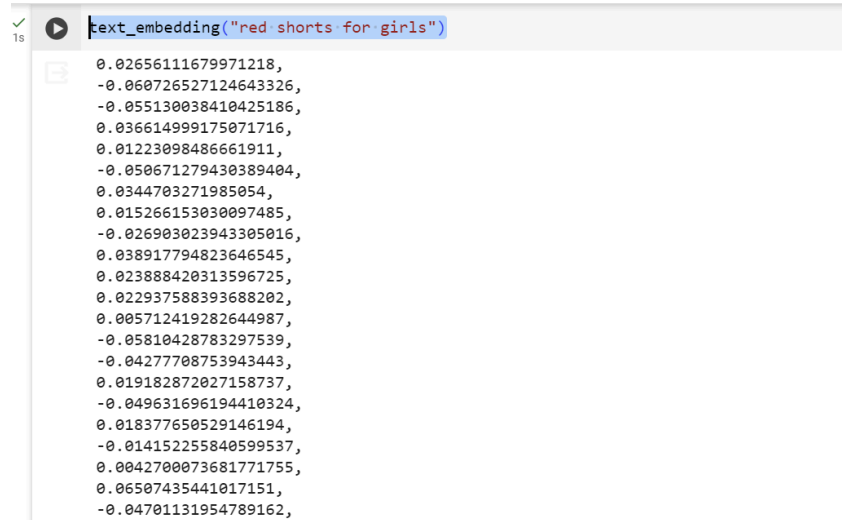
下列方法會叫用文字嵌入模型，並傳回使用者輸入文字的向量嵌入項目：

```
def text_embedding(content) -> list:
    """Text embedding with a Large Language Model."""
    model = TextEmbeddingModel.from_pretrained("textembedding-gecko@latest")
    embeddings = model.get_embeddings(content)
    for embedding in embeddings:
        vector = embedding.values
        #print(f"Length of Embedding Vector: {len(vector)}")
    return vector
```

測試

```
text_embedding("red shorts for girls")
```

輸出內容應如下所示 (請注意，圖片已在高度方面經過裁剪，因此您無法看到完整的向量回應)：



```
text_embedding("red shorts for girls")
```

```
0.02656111679971218,  
-0.060726527124643326,  
-0.055130038410425186,  
0.036614999175071716,  
0.01223098486661911,  
-0.050671279430389404,  
0.0344703271985054,  
0.015266153030097485,  
-0.026903023943305016,  
0.038917794823646545,  
0.023888420313596725,  
0.022937588393688202,  
0.005712419282644987,  
-0.05810428783297539,  
-0.04277708753943443,  
0.019182872027158737,  
-0.049631696194410324,  
0.018377650529146194,  
-0.014152255840599537,  
0.0042700073681771755,  
0.06507435441017151,  
-0.04701131954789162,
```

4. 宣告已部署的索引 ID 和端點 ID

```
from google.cloud import aiplatform  
DEPLOYED_INDEX_ID = "spanner_vector1_1702366982123"
```

```
#Vector Search Endpoint  
index_endpoint = aiplatform.MatchingEngineIndexEndpoint('projects/273845608377/locations/us-  
central1/indexEndpoints/2021628049526620160')
```

5. 定義 Vector Search 方法，呼叫索引端點，並顯示與使用者輸入內容對應的嵌入回應最接近的 10 個相符項目。

在 Vector Search 的下列方法定義中，請注意，系統會叫用 `find_neighbors` 方法，找出 10 個最鄰近的向量。

```

def vector_search(content) -> list:
    result = text_embedding(content)
    #call_vector_search_api(content)
    index_endpoint =
aiplatform.MatchingEngineIndexEndpoint('projects/273845608377/locations/us-
central1/indexEndpoints/2021628049526620160')
    # run query
    response = index_endpoint.find_neighbors(
        deployed_index_id = DEPLOYED_INDEX_ID,
        queries = [result],
        num_neighbors = 10
    )
    out = []
    # show the results
    for idx, neighbor in enumerate(response[0]):
        print(f"{neighbor.distance:.2f} {spanner_read_data(neighbor.id)}")
        out.append(f"{spanner_read_data(neighbor.id)}")
    return out

```

您也會注意到對 `spanner_read_data` 方法的呼叫。請參閱下一個步驟。

6. 定義 Spanner 讀取資料方法實作，叫用 `execute_sql` 方法，擷取與上一步傳回的最近鄰項向量 ID 對應的圖片。

```

!pip install google-cloud-spanner==3.36.0

from google.cloud import spanner

instance_id = "spanner-vertex"
database_id = "spanner-vertex-embeddings"
projectId = PROJECT_ID
client = spanner.Client()
client.project = projectId
instance = client.instance(instance_id)
database = instance.database(database_id)

def spanner_read_data(id):
    query = "SELECT uri FROM apparels where id = " + id
    outputs = []
    with database.snapshot() as snapshot:
        results = snapshot.execute_sql(query)

        for row in results:
            #print(row)
            #output = "ID: {}, CONTENT: {}, URI: {}".format(*row)
            output = "{}".format(*row)
            outputs.append(output)

    return "\n".join(outputs)

```

系統應會傳回與所選向量對應的圖片網址。

7. 最後，讓我們在使用者介面中整合各個部分，並觸發向量搜尋程序

```
from PIL import Image

def call_search(query):
    response = vector_search(query)
    return response

input_text = gr.Textbox(label="Enter your query. Examples: Girls Tops White Casual, Green t-shirt girls, jeans shorts, denim skirt etc.")
output_texts = [gr.Image(label="") for i in range(10)]
demo = gr.Interface(fn=call_search, inputs=input_text, outputs=output_texts, live=True)
resp = demo.launch(share = True)
```

您應該會看到如下所示的結果：



圖片：[連結](#)

如要觀看結果影片，請按[這裡](#)。

7. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本文章所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後按一下「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關閉) 刪除專案。
 4. 如果不想刪除專案，請刪除 Spanner 執行個體，方法是前往您為這個專案建立的執行個體，然後按一下執行個體總覽頁面右上角的「DELETE INSTANCE」(刪除執行個體) 按鈕。
 5. 您也可以前往 Vector Search 搜尋索引，取消部署端點和索引，然後刪除索引。
-

8. 結論

恭喜！您已成功完成 Spanner - Vertex 向量搜尋實作，方法如下：

1. 為從 Spanner 資料庫取得來源的應用程式建立 Spanner 資料來源和嵌入內容。
2. 建立 Vector Search 資料庫索引。
3. 使用 Dataflow 和 Workflow 工作，將 Spanner 的向量資料整合至 Vector Search。
4. 將索引部署至端點。
5. 最後，在以 Python 實作的 Vertex AI SDK 中，對使用者輸入內容叫用 Vector Search。

歡迎根據自己的用途擴充實作項目，或使用新功能改良目前的用途。如要進一步瞭解 Spanner 的機器學習功能，請參閱[這篇文章](#)。
